

The code, explained line by line

Four files. 541 lines. Everything the agent does, and why each line is there.

Abdelrahman Hamdy

Marketing · credit card retention

How to read this

Do not read it front to back in one sitting. Take one file at a time. For each one: read what it does, read the code, then read the notes beside it.

The notes only cover lines that are not obvious. If a line is missing from the notes, it means it says what it does.

SECTION ONE

The map

Four files. Only one of them does any thinking.

File	Lines	Job
<code>agent.py</code>	158	Talks to you. Collects the customer's details.
<code>tool.py</code>	161	Does all the thinking. Every number comes from here.
<code>offers.py</code>	101	The twelve offers, and the rule that picks one.
<code>churn_features.py</code>	121	Plumbing the saved model needs. Never edit it.

Which way things point

```
1 agent.py          you talk to this
2   |
3   v
4 tool.py           does all the thinking
5   |
6   +--> offers.py      picks the offer
7   +--> churn_features.py lets the model file open
8   +--> models/churn_model.pkl
```

Nothing points backwards. tool.py has never heard of agent.py, which is why you can test the tool on its own.

One sentence to memorise

agent.py talks. tool.py thinks. offers.py chooses. churn_features.py is plumbing.

Where to look when something is wrong

wrong offer	offers.py — the rules in <code>pick_offer</code>
wrong risk	tool.py — the <code>score</code> function
wrong value	tool.py — the <code>yearly_value</code> function
agent confused	agent.py — <code>read_details</code> or <code>next_question</code>
model won't load	churn_features.py must exist, and models/churn_model.pkl must exist

offers.py

Twelve offers the bank can give a customer, and one function that picks the right one. There is no cleverness in this file at all — which is why you should read it first.

Part 1 — the offers

Each offer is a **dictionary**: curly brackets holding labelled values. `RATE_CUT["cost"]` gives you 500. That is all a dictionary is — a labelled box.

```
11 NO_OFFER = {
12     "name": "No offer",
13     "cost": 0,
14     "fixes": "Nothing. This customer is not worth spending money on.",
15 }
16
17 RATE_CUT = {
18     "name": "Lower the interest on their balance for 6 months",
19     "cost": 500,
20     "fixes": "They are struggling with what the card costs them each month.",
21 }
22
23 INSTALMENTS = {
24     "name": "Turn their balance into 0% instalments",
25     "cost": 500,
26     "fixes": "They are stuck with a balance they cannot clear.",
27 }
28
29 SALARY_BONUS = {
30     "name": "Cash bonus for moving their salary to us",
31     "cost": 1000,
32     "fixes": "We are their side bank, not their main one.",
33 }
34
35 FEE_WAIVER = {
36     "name": "Waive the annual fee for a year",
37     "cost": 600,
38     "fixes": "They do not use the card enough to justify paying for it.",
39 }
40
41 CASHBACK = {
42     "name": "Extra cashback in one category for 3 months",
43     "cost": 400,
44     "fixes": "Their other card gives them a better reason to use it.",
45 }
46
47 FAMILY_CARDS = {
48     "name": "Free extra cards for their family",
49     "cost": 150,
50     "fixes": "The card is personal, so nothing in the household depends on it.",
51 }
52
53 MERCHANT_PLANS = {
54     "name": "0% instalments at our partner shops",
55     "cost": 250,
56     "fixes": "The card offers nothing their other card does not.",
57 }
```

name	What the employee will be told to offer.
cost	What it costs the bank. These are placeholders. The bank sets the real ones.
fixes	The problem this offer solves. The agent reads this out loud, which is how it explains <i>why</i> it chose the offer.

Part 2 — choosing one

Read this top to bottom. **The first rule that matches wins, and nothing below it runs.** That is the whole logic.

```
60 def pick_offer(customer, kind, yearly_value):
61     """Choose one offer for this customer.
62
63     customer      - the raw details
64     kind          - "distressed" or "drifting", from tool.py
65     yearly_value  - what they are worth to the bank per year, in EGP
66
67     Read it top to bottom. The first rule that matches wins.
68     """
69
70     # Rule 1. Too small to be worth anything. Nothing else matters.
71     if yearly_value < 800:
72         return NO_OFFER
73
74     # Rule 2. A money problem. Make the card cheaper, never ask them to spend more.
75     if kind == "distressed":
76         if customer["outstanding_loan_balance"] > 0:
77             return INSTALMENTS
78         return RATE_CUT
79
80     # Rule 3. A drifting customer. Give them a reason to use the card again.
81     if customer["salary_lands_in_bank"] == 0:
82         return SALARY_BONUS
83
84     if customer["has_other_credit_cards"] == 1:
85         return CASHBACK
86
87     if customer["married"] == 1 or customer["has_dependents"] == 1:
88         return FAMILY_CARDS
89
90     if recent_spending(customer) < 900:
91         return FEE_WAIVER
92
93     return MERCHANT_PLANS
```

return Means *stop here and hand this back*. Nothing after a return runs.

Rule 1 Worth less than 800 EGP a year? No offer, and we never even look at what kind of customer they are. If they are not worth the money, nothing else matters.

Rule 2 A money problem. Make the card cheaper. **Never** ask a struggling customer to spend more — that would be the wrong offer and arguably a harmful one.

Rule 3 A drifting customer, checked in a deliberate order: salary first because it is the strongest finding in the analysis, then a rival card, then family.

order The order *is* the logic. Moving a rule up or down changes the answer.

```
96 def recent_spending(customer):
97     """Average card spending over the last three months."""
98     last_three = [customer["purchase_month_4"],
99                  customer["purchase_month_5"],
100                 customer["purchase_month_6"]]
101     return sum(last_three) / 3
```

Months 4, 5 and 6 are the most recent three. Month 1 is the oldest.

tool.py

Four questions, four functions. This is the only file that produces a number.

<code>score()</code>	Will they close the card, and why?
<code>yearly_value()</code>	What are they worth to the bank per year?
<code>kind_of()</code>	A money problem, or a boredom problem?
<code>assess()</code>	All of the above, plus the offer, plus whether it pays.

The settings at the top

```

24 MODEL_FILE = os.path.join(os.path.dirname(__file__), "..", "models", "churn_model.pkl")
25
26
27 # The four rates the bank must set. PLACEHOLDERS for now.
28 CUT_OF_SPENDING = 0.018      # what the bank keeps from every purchase
29 INTEREST_RATE = 0.30         # yearly interest on unpaid balance
30 LOAN_MARGIN = 0.05           # yearly margin on a loan
31 DEPOSIT_MARGIN = 0.03        # yearly margin on salary held here
32
33 # Our guess at how often an offer actually works. A guess, not a measurement.
34 CHANCE_OFFER_WORKS = {"distressed": 0.40, "drifting": 0.25}
```

four rates	Placeholders. Ask your manager for the real ones. Every value figure in the project moves with these four numbers, which is why they sit together in one place.
CHANCE_...	A guess, not a measurement. Measuring it properly needs an experiment where some at-risk customers are deliberately left alone. Say this before anyone asks.

yearly_value — the money

Four ways a bank makes money from a card customer, added together.

```
74 def yearly_value(customer):
75     """What is this customer worth to the bank per year, in EGP?
76
77     Four ways the bank makes money from a card customer, added together.
78     """
79     purchases = [customer[f"purchase_month_{i}"] for i in range(1, 7)]
80     payments = [customer[f"payment_month_{i}"] for i in range(1, 7)]
81
82     spent_per_year = sum(purchases) * 2          # six months doubled
83     share_paid_back = sum(payments) / sum(purchases)
84     unpaid = spent_per_year * (1 - share_paid_back)
85
86     from_spending = spent_per_year * CUT_OF_SPENDING
87     from_interest = unpaid * INTEREST_RATE
88     from_loan = customer["outstanding_loan_balance"] * LOAN_MARGIN
89     from_salary = 0
90     if customer["salary_lands_in_bank"] == 1:
91         from_salary = customer["salary"] * 12 * DEPOSIT_MARGIN
92
93     return round(from_spending + from_interest + from_loan + from_salary)
94
95
96 # -----
```

purchases = [...]	Collects the six monthly figures into a list. <code>f"purchase_month_{i}"</code> builds the name — when <code>i</code> is 3 it becomes <code>purchase_month_3</code> . <code>range(1, 7)</code> counts 1 to 6; it stops before 7, not at it.
* 2	We only have six months of data, so double it to estimate a year.
share_paid	Spent 100, repaid 76 gives 0.76.
unpaid	1 minus 0.76 is 0.24, so 24% of their spending stays on the card. This is the part the bank charges interest on, and it is the biggest source of money.
from_salary	Zero unless their salary is paid to us. This is exactly why the salary transfer offer exists — it turns a zero into a real number.

A customer who always pays in full is worth less to the bank than one who carries a balance. Uncomfortable, but true, and it falls straight out of these four lines.

kind_of — which problem is it?

```
97 def kind_of(customer):
98     """A money problem, or a boredom problem?
99
100     This decides which offers are even allowed. Asking a struggling customer to
101     spend more would be the wrong offer, and possibly a harmful one.
102     """
103     purchases = [customer[f"purchase_month_{i}"] for i in range(1, 7)]
104     payments = [customer[f"payment_month_{i}"] for i in range(1, 7)]
105     share_paid_back = sum(payments) / sum(purchases)
106
107     struggling = (share_paid_back < 0.80
108                  or customer["missed_loan_payment_ever"] == 1
109                  or customer["outstanding_loan_balance"] > 8000)
110
111     return "distressed" if struggling else "drifting"
112
113
114 # -----
```

or

Any one of the three is enough to count as struggling. Had it been **and**, all three would need to be true and almost nobody would qualify — most struggling customers would be misread as merely bored and offered the wrong thing.

why it matters

This one word decides which offers are even allowed.

assess — putting it together

```
115 def assess(customer):
116     """Everything about one customer, in one dictionary."""
117     risk, threshold, reasons = score(customer)
118     value = yearly_value(customer)
119     kind = kind_of(customer)
120     offer = pick_offer(customer, kind, value)
121
122     # Is the offer worth making?
123     # We gain (risk x chance it works x value). We pay the cost.
124     chance = CHANCE_OFFER_WORKS[kind]
125     expected_gain = risk * chance * value
126     worth_it = expected_gain > offer["cost"]
127
128     return {
129         "risk_out_of_100": round(risk * 100),
130         "acting_line": round(threshold * 100),
131         "will_close": risk >= threshold,
132         "reasons": reasons,
133         "worth_per_year": value,
134         "customer_kind": kind,
135         "offer": offer["name"],
136         "offer_fixes": offer["fixes"],
137         "offer_cost": offer["cost"],
138         "expected_gain": round(expected_gain),
139         "make_the_offer": worth_it and risk >= threshold,
140     }
141
142
143 # -----
```


expected_gain	Three numbers multiplied: how likely they leave, how likely an offer works, what they are worth. For the test customer: $0.49 \times 0.40 \times 2719$, about 533.
worth_it	533 against a cost of 500. It passes, but only just.
make_the_offer	Also requires the risk to be above the line. A low-risk customer might still clear the money test, and we should not chase someone who was never leaving.

score — the only hard part

Take your time here. This is the one piece of the project that is genuinely difficult, and it is the piece your manager is most likely to ask about.

```
47 def score(customer):
48     """Will they close the card? Returns the risk and the top three reasons."""
49     saved = load_model()
50     model = saved["model"]
51
52     one_row = pd.DataFrame([customer])
53     risk = float(model.predict_proba(one_row)[0, 1])
54
55     # WHY. The model is a weighted sum: every fact is multiplied by a weight and
56     # added up. So the reason a customer scored high is just that list of
57     # multiplications, biggest first.
58     prepared = model[:-1].transform(one_row)[0]
59     weights = model[-1].coef_[0]
60     names = model.named_steps["features"].columns_
61
62     effects = prepared * weights
63     biggest = sorted(range(len(effects)), key=lambda i: effects[i], reverse=True)
64
65     reasons = []
66     for i in biggest[:3]:
67         high_or_low = "high" if prepared[i] > 0 else "low"
68         reasons.append(f"their {names[i]} is {high_or_low}")
69
70     return risk, saved["threshold"], reasons
71
72
73 # -----
```

<code>pd.DataFrame</code>	The model was trained on a table of 8,500 customers, so it expects a table. This makes a table with one row in it.
<code>[0, 1]</code>	<code>predict_proba</code> returns two numbers: the chance they stay, then the chance they leave. Row 0, column 1 is the chance they leave.
<code>model[:-1]</code>	Every step <i>except</i> the last — the parts that prepare the numbers.
<code>model[-1]</code>	The last step: the scorecard itself.
<code>.coef_</code>	The weights it learned during training.
<code>prepared*weights</code>	This is the whole idea. A logistic regression works like a scorecard: every fact is multiplied by a weight, and the results are added up. So this list of multiplications <i>is</i> the reason the customer scored what they scored.

Why there is no SHAP here

The original plan said use SHAP. SHAP exists to *estimate* reasons for models too complicated to read directly — a forest of hundreds of trees, for example. A scorecard can be read directly. We show the model's own arithmetic instead of an approximation of it: simpler, exact, and explainable in one sentence.

agent.py

You type. The language model reads your words. Our code keeps the details and decides what to ask for next.

The list of things we need

```

26  NEEDED = {
27      "age": "their age",
28      "married": "whether they are married (1 or 0)",
29      "has_dependents": "whether they have children (1 or 0)",
30      "employment_sector": "where they work: Private, Government or Self-Employed",
31      "salary": "monthly salary",
32      "salary_lands_in_bank": "whether their salary is paid to us (1 or 0)",
33      "loyalty_years": "years with the bank",
34      "iscore": "their I-Score credit score",
35      "has_other_credit_cards": "whether they have a card elsewhere (1 or 0)",
36      "had_loan_ever": "whether they ever took a loan (1 or 0)",
37      "number_of_loans": "how many loans",
38      "outstanding_loan_balance": "how much they still owe",
39      "missed_loan_payment_ever": "whether they ever missed a payment (1 or 0)",
40  }
41  for i in range(1, 7):
42      NEEDED[f"purchase_month_{i}"] = f"card spending in month {i}"
43  for i in range(1, 7):
44      NEEDED[f"payment_month_{i}"] = f"card repayment in month {i}"

```

NEEDED

The 25 details the churn model requires, each with the plain-English way to ask for it.

the two loops

Rather than typing out twelve near-identical lines, we build them. This is the same trick as in `yearly_value`.

The form we give the language model

```

48  TOOL_FORM = {
49      "type": "function",
50      "function": {
51          "name": "assess_customer",
52          "description": "Assess a credit card customer. Fill in every value you can find.",
53          "parameters": {
54              "type": "object",
55              "properties": {name: {"type": "string", "description": ask}
56                          for name, ask in NEEDED.items()},
57              "required": [],
58          },
59      },
60  }

```

This is a description of a function, not the function itself. We hand it to the model so it knows what shape of answer to give back. The real work is still done by our Python.

Reading details out of your words

```
78 def read_details(notes, known):
79     """Ask the model to pull customer details out of what was typed.
80
81     Anything it finds goes into `known`. Anything it misses, we already have.
82     We keep the details, not the model.
83     """
84     reply = ask_model(
85         [{"role": "user",
86          "content": "Call assess_customer with every value you can find in these "
87                   "notes about one bank customer. Do not guess. Notes:\n" + notes}],
88         tools=[TOOL_FORM],
89     )
90
91     for call in reply["message"].get("tool_calls") or []:
92         found = call["function"]["arguments"]
93         if isinstance(found, str):
94             found = json.loads(found)
95         for name in NEEDED:
96             if found.get(name) not in (None, "", "unknown"):
97                 known[name] = found[name]
```

This is the most important design decision in the project.

<code>known[name] =</code>	Whatever the model found is copied into known . known belongs to us. It only ever grows. The model never holds it.
<code>why</code>	A small model forgets. Ask it to carry 25 numbers across five messages and it will drop some and invent others. So we do not ask. We keep them, and the model only reads.
<code>the risk that remains</code>	If the model invents a detail that was never mentioned, it lands in known and produces a wrong answer. That is why the prompt says <i>do not guess</i> . If asked about weaknesses, name this one.

Deciding what to ask next

```
100 def next_question(known):
101     """What should we ask for next? At most four things."""
102     missing = [name for name in NEEDED if name not in known]
103
104     spending = [f"purchase_month_{i}" for i in range(1, 7)]
105     repaying = [f"payment_month_{i}" for i in range(1, 7)]
106
107     if all(m in missing for m in spending):
108         return "What was their card spending each month for the last 6 months?"
109     if all(m in missing for m in repaying):
110         return "And their repayments for those same 6 months?"
111
112     asking = missing[:4]
113     return "I still need: " + ", ".join(NEEDED[m] for m in asking)
```

No language model here. Our code decides what to ask for. The first version of this project let the model decide, and it stopped calling the tool altogether and began answering as though it were sitting an exam. The fix was not a bigger model — it was giving the model less to do.

The loop

```
128 def main():
129     print("Describe a customer. Type 'quit' to stop, 'new' to start over.\n")
130
131     known = {}
132     notes = ""
133
134     while True:
135         typed = input("You: ").strip()
136
137         if typed.lower() in ("quit", "exit"):
138             break
139         if typed.lower() == "new":
140             known, notes = {}, ""
141             print("\nAgent: Starting fresh.\n")
142             continue
143         if not typed:
144             continue
145
146         notes = notes + "\n" + typed
147         read_details(notes, known)
148
149         if len(known) < len(NEEDED):
150             print(f"\nAgent: {next_question(known)}")
151         else:
152             print("\nAgent:", explain(assess(known)))
153
154     print(f"      [{len(known)} of {len(NEEDED)} details]\n")
```

while True	Repeat forever until something breaks out.
notes = notes +	We keep everything you have said and resend all of it each time. If the model missed a detail on turn one, it gets another chance on turn two with the full picture.
len(known)	Still missing something? Ask. Otherwise do the work.
two model calls	The model is used in exactly two places in this whole file: <code>read_details</code> , which reads English, and <code>explain</code> , which writes it. Everything else is plain Python.

churn_features.py

Do not read this file's code. Read why it exists, then leave it alone.

Why it has to be there

When the notebook saved `churn_model.pkl`, it saved the model's *numbers* but not its *code*. Inside that file is a note which says, in effect: *I am built from a thing called ChurnFeatures*.

So anything opening that file must already have ChurnFeatures defined. In Colab it was in the notebook. On your computer it lives in this file. That is the only reason it exists.

It is a copy, and it must stay a copy.

If you change one line in it, the model will still load and still produce numbers. They will just be **different numbers**, with nothing to warn you. That is the worst kind of bug there is — the kind that does not announce itself.

The one function in it that is ours

```
105 def register_for_unpickling():
106     """Make the saved file loadable.
107
108     The model was saved from a notebook, so inside churn_model.pkl the two
109     classes are recorded as living in a place called "__main__" - which means
110     "whatever file is being run right now".
111
112     This puts them there, so joblib finds them where it expects to.
113     """
114     import __main__
115     __main__.ChurnFeatures = ChurnFeatures
116     __main__.ClipExtremes = ClipExtremes
117     __main__.PURCHASE = PURCHASE
118     __main__.PAYMENT = PAYMENT
119     __main__.MONTHS = MONTHS
120     __main__.PARTNERS = PARTNERS
121     __main__.NEVER_USED = NEVER_USED
```

This puts the two classes where the saved file expects to find them. Without it you get the error you saw the first time: "Can't get attribute ChurnFeatures".

Questions you will be asked

Practise these out loud. Not reading them — saying them.

“Walk me through what happens when someone types a customer in.”

The model reads my words and fills in a form. My code stores what it found. If anything is missing, my code asks for it. Once all 25 details are there, my code runs the trained model, works out what the customer is worth, picks an offer, and the language model turns that into a sentence.

“What does the AI actually decide?”

Nothing that matters. It reads English and it writes English. Every number — the risk, the value, the offer — comes from my own code. If the model invented a number it would not reach the answer, because it never produces one.

“Why no framework?”

The loop is about forty lines. A framework would hide it, and then every question about how it behaves would have the answer “the framework does it”. This way I can point at the line.

“Why not SHAP?”

SHAP estimates reasons for models too complicated to read directly. Mine is a logistic regression — a scorecard. I can read the actual multiplications, so I do not need an estimate of them.

“What is weakest about this?”

Three things. The chance an offer works is a guess, not a measurement. The offer costs are placeholders until the bank gives me real ones. And the language model could in principle invent a customer detail, which is why I keep the details in my own code rather than letting it hold them.

“Could it be simpler?”

It is four files and 541 lines. It was 1,425. I cut the classes, the rules engine and the extra layers. What is left is the trained model, four money calculations, eight if-statements for the offers, and a loop.

When you are stuck

Add a `print()` and run it. That is not a beginner's trick — it is what everyone does. If you cannot tell what a line does, print the thing just before it and the thing just after it.

```
1 print("HERE:", the_thing)
```